

Шелл-код

Текущая версия страницы пока [не проверялась](#) опытными участниками и может значительно отличаться от [версии, проверенной 16 июля 2025 года](#); проверки требуют [2 правки](#).

Шелл-код ([англ. shellcode](#), код запуска оболочки) — это [двоичный исполняемый код](#), который обычно передаёт управление командному процессору, например `'/bin/sh'` в [Unix shell](#), `'command.com'` в [MS-DOS](#) и `'cmd.exe'` в [операционных системах Microsoft Windows](#). Шелл-код может быть использован как полезная нагрузка [эксплойта](#), обеспечивающая [взломщику](#) доступ к [командной оболочке](#) ([англ. shell](#)) в [компьютерной](#) системе.

При эксплуатации удаленной уязвимости шелл-код может открывать заранее заданный порт [TCP](#) уязвимого компьютера, через который будет осуществляться дальнейший доступ к командной оболочке, такой код называется привязывающим к порту ([англ. port binding shellcode](#)). Если шелл-код осуществляет подключение к порту компьютера атакующего, что производится с целью обхода [брандмауэра](#) или [NAT](#), то такой код называется обратной оболочкой ([англ. reverse shell shellcode](#)).

Принцип работы

Шелл-код обычно внедряется в память эксплуатируемой программы, после чего на него передается управление путём переполнения [стека](#), или при [переполнении буфера в куче](#), или используя [атаки форматной строки](#). Передача управления шелл-коду осуществляется перезаписью адреса возврата в стеке адресом внедрённого шелл-кода, перезаписью адресов вызываемых функций или изменением обработчиков прерываний. Результатом этого является выполнение шелл-кода, который открывает командную строку для использования взломщиком.

Обнаружение

[Взломщики](#) пишут шелл-коды, часто используя приёмы, скрывающие их атаку. Они часто пытаются выяснить, как [системы обнаружения вторжений](#) (СОВ) распознают любую входящую атаку. Типичная [СОВ](#) обычно просматривает все входящие пакеты в поисках структуры, специфичной для шелл-кода (часто большой массив [мусорных](#) кодов, в простейшем случае [NOP-ов](#)); если она находит такую структуру, пакет уничтожается до того, как он достигнет своей цели. Слабая позиция СОВ в данном случае состоит в том, что она не осуществляет действительно хороший поиск, иначе он займёт слишком много времени и, таким образом, замедлит соединение с интернетом.

Шелл-код почти всегда содержит строку с именем **оболочки**. Все входящие пакеты, содержащие такую строку, всегда рассматриваются как подозрительные в глазах СОВ. Также некоторые приложения не принимают неалфавитно-цифровой ввод (они не принимают символов, выходящих за рамки набора a-z, A-Z, 0-9 и нескольких других символов.)

Для прохождения через все эти меры, направленные против вторжения, взломщики используют **шифрование**, **сагомодифицирующийся код**, **полиморфный код** и **алфавитно-цифровой код**.

Внутреннее устройство и классическая структура шелл-кода

По своей сути, шелл-код — это минималистичная, самодостаточная машинная программа, написанная на **ассемблере** и часто представленная в виде строки **байт-кода** (опкодов). Его цель — выполнить строго определённую задачу (например, запуск оболочки) с использованием минимального объема памяти и без зависимостей.

Канонический пример: шелл-код для **Linux x86** (`execve(«/bin/sh»)`)

Это классический учебный пример. Его задача — выполнить системный вызов `execve` с аргументом `/bin/sh`.

Логика на **C**

```
execve("/bin/sh", NULL, NULL);
```

Разбор на ассемблере (AT&T syntax)

Обнуление регистров

Сначала нужно обнулить регистры, которые будут использоваться как аргументы, чтобы избежать нулевых байт.

```
xorl    %eax, %eax        # EAX = 0 (системный вызов execve)
xorl    %edx, %edx        # EDX = 0 (envp = NULL)
```

Помещение строки «/bin/sh» в стек

Стек работает как **LIFO**, поэтому строку кладут задом наперед. Также важно завершить её нулевым байтом.

```
pushl    %edx                # Поместить нулевой байт (конец строки)
pushl    $0x68732f2f        # Поместить "hs//" (//sh, второй / для
выравнивания)
pushl    $0x6e69622f        # Поместить "nib/" (/bin)
```

Теперь в стеке лежит строка `/bin//sh\0`, а регистр `%esp` (указатель стека) указывает на её начало.

Подготовка аргументов для `execve`

```
movl     %esp, %ebx          # EBX = указатель на строку "/bin//sh\0"
(argv[0])
pushl    %edx                # argv[1] = NULL
pushl    %ebx                # argv[0] = указатель на строку
movl     %esp, %ecx          # ECX = указатель на массив argv[]
```

Системный вызов `execve` в Linux x86 принимает аргументы: `eax` (номер вызова), `ebx` (путь к файлу), `ecx` (argv), `edx` (envp).

Выполнение системного вызова

```
movb     $0xb, %al           # Номер системного вызова execve = 11 (0xb)
int      $0x80               # Прерывание для вызова ядра
```

Итоговый байт-код (hex)

```
31 c0 31 d2 52 68 2f 2f 73 68 68 2f 62 69 6e 89 e3 52 53 89 e1 b0
0b cd 80
```

Это и есть чистый шелл-код — последовательность инструкций [процессора](#). Его можно вставить в буфер C-программы как массив символов и передать на него выполнение.

См. также

- [Переполнение буфера](#)
- [Переполнение кучи](#)
- [Информационная безопасность](#)

- [Язык ассемблера](#)
- [Heap spraying](#)

Ссылки

- *Steve Hanna*. Shellcoding for Linux and Windows Tutorial with example windows and linux shellcode (<https://web.archive.org/web/20050728094519/http://www.vividmachines.com/shellcode/shellcode.html>) (англ.) (2004). Дата обращения: 30 сентября 2007. Архивировано из оригинала (<http://www.vividmachines.com/shellcode/shellcode.html>) 28 июля 2005 года.
- Designing Shellcode Demystified (<https://web.archive.org/web/20050829191606/http://www.enderunix.org/docs/en/sc-en.txt>) (англ.). Дата обращения: 28 июля 2005. Архивировано из оригинала (<http://www.enderunix.org/docs/en/sc-en.txt>) 29 августа 2005 года.

[Сообщить об ошибке](#)